

ЛАБОРАТОРНА РОБОТА №1.

ФРОНТЕНД

Мета роботи: Створити базову сторінку застосунку з HTML-формою та відображенням даних у таблиці/списку. Навчитися працювати з DOM (події, читання значень полів, динамічний рендеринг). Реалізувати мінімальну валідацію введення.

Теоретичні відомості

Структура простої веб-сторінки

У межах першої лабораторної роботи достатньо розуміти веб-сторінку як набір статичних файлів, які браузер завантажує та інтерпретує:

- index.html – опис структури сторінки (розмітка): які на ній є блоки, форма, таблиця/список тощо.
- styles.css – оформлення: шрифти, відступи, вирівнювання, кольори, розкладка елементів.
- app.js – поведінка: читання даних з форми, валідація, додавання записів у масив, динамічне оновлення таблиці/списку, обробка натискань кнопок.

Цей поділ важливий з двох причин:

1. Зручність підтримки: розмітка, стилі та логіка не «перемішані», легше знаходити і виправляти помилки.
2. Масштабування проекту: у наступних лабораторних студенти будуть розширювати один і той самий застосунок; чиста структура файлів спрощує інкрементальну розробку.

Мінімальний каркас HTML-документа

Будь-яка сучасна HTML-сторінка має стандартний «скелет»:

- `<!doctype html>` повідомляє браузеру, що документ написаний у HTML5.
- `<html>` – кореневий елемент сторінки.

- `<head>` містить службову інформацію: кодування, заголовки, підключення стилів/скриптів.
- `<body>` містить те, що користувач бачить: заголовки, форму, таблицю тощо.

Приклад мінімальної структури:

```
<!doctype html>
<html lang="uk">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>ЛР1 – Мій застосунок</title>

  <link rel="stylesheet" href="styles.css" />
  <script src="app.js" defer></script>
</head>
<body>
  <h1>Мій застосунок (варіант N)</h1>

  <!-- Тут буде форма -->
  <!-- Тут буде таблиця/список -->
</body>
</html>
```

Зверніть увагу на кілька моментів:

- `lang="uk"` – мова сторінки. Це корисно для доступності та коректної роботи деяких браузерних функцій.
- `<meta charset="utf-8">` – кодування UTF-8, щоб коректно відображались українські символи.
- `<meta name="viewport" ...>` – базове технічне налаштування для коректного масштабування на різних екранах.

Підключення CSS

Найпоширеніший спосіб підключити стилі – тег `<link>` у секції `<head>`:

```
<link rel="stylesheet" href="styles.css" />
```

Файл `styles.css` має бути в тій самій папці, що й `index.html` (або шлях має відповідати вашій структурі). У цьому файлі описують стилі для елементів (наприклад, форма, таблиця, повідомлення про помилки).

Підключення JavaScript

Скрипт підключають через тег `<script>`. Для навчального проекту рекомендований варіант:

```
<script src="app.js" defer></script>
```

Навіщо потрібен defer

Без додаткових параметрів браузер може почати виконувати JavaScript ще до того, як повністю «побудує» DOM-дерево сторінки. Це часто призводить до типових помилок, коли код намагається знайти елемент на сторінці, а його ще «немає» у DOM.

Атрибут `defer` означає:

- браузер завантажить файл `app.js`,
- але виконає його лише після того, як HTML буде розібраний і DOM сформований.

Практично це дозволяє безпечно писати код на кшталт:

1. знайти форму через `document.querySelector(...)`,
2. підписатися на `submit`,
3. оновити таблицю/список,

не переживаючи, що елементи ще не існують у момент виконання скрипта.

Альтернатива (яку теж можна зустріти)

Інший поширений підхід – підключати `<script>` в кінці `body`, щоб HTML гарантовано вже був завантажений. У межах лабораторної роботи обидва підходи допустимі, але `defer` в `head` зазвичай виглядає акуратніше та дисциплінує структуру.

Типова структура папки проекту

Рекомендована мінімальна структура:

```
my-lab1/  
  index.html  
  styles.css  
  app.js
```

У наступних лабораторних, коли з'являться додаткові файли (наприклад, окремі модулі JS, картинки, тощо), структуру можна буде розширити, але на старті важливо мати просту й зрозумілу основу.

Принцип «від статики до динаміки» (корисна модель мислення)

Для виконання першої лабораторної роботи зручно мислити так:

1. Спочатку робимо статичну розмітку: заголовок, форма, порожня таблиця/список.
2. Потім додаємо CSS, щоб сторінка стала читабельною.
3. Лише після цього додаємо JavaScript:
4. зчитати введення,
5. перевірити його,
6. оновити дані,
7. перемалювати таблицю/список.

Цей порядок зменшує кількість помилок і дозволяє швидше локалізувати проблему: якщо «не працює» – зрозуміло, де шукати (розмітка, стилі чи логіка).

HTML: базові теги та семантика для форми і таблиці

У даній лабораторній роботі HTML потрібен як «каркас» застосунку: в ньому описується, які елементи є на сторінці, а JavaScript потім керує їхньою поведінкою (читає дані, виводить список, реагує на натискання кнопок). На цьому етапі важливо не «вивчити весь HTML», а впевнено користуватися невеликим набором тегів, яких достатньо для форми та таблиці.

Блоки та контейнери: як «скласти» сторінку

Найчастіше сторінку розбивають на логічні секції:

- `<header>` – заголовок сторінки (назва застосунку, варіант).
- `<main>` – основний вміст: форма та список/таблиця.
- `<section>` – окрема логічна частина (наприклад, «Додавання запису» і «Список записів»).
- `<div>` – універсальний контейнер «на всі випадки», якщо не підходить жоден семантичний тег.

Приклад базового каркаса:

```
<header>
  <h1>Мій застосунок (варіант N)</h1>
</header>

<main>
  <section id="create-section">
    <h2>Додати запис</h2>
    <!-- форма -->
  </section>

  <section id="list-section">
    <h2>Список записів</h2>
    <!-- таблиця або список -->
  </section>
</main>
```

Семантичні теги (header, main, section) не є «обов'язковими», але роблять розмітку більш читабельною.

Текстові теги, які реально потрібні

Для даної роботи достатньо:

- <h1>, <h2>, <h3> – заголовки різних рівнів.
- <p> – абзац тексту (пояснення, інструкція, повідомлення).
- – невеликий «інлайновий» елемент для частин тексту (наприклад, підказка).
- , – виділення важливих фрагментів (жирний / курсив).

Наприклад, коротка інструкція під заголовком:

```
<p>Заповніть форму і натисніть <strong>Додати</strong>. Запис з'явиться у таблиці нижче.</p>
```

Списки: коли таблиця не обов'язкова

Список – це простий спосіб показати дані, якщо у полів небагато або хочеться «картки».

- – маркований список.
- – нумерований список.
- – елемент списку.

Приклад:

```
<ul id="itemsList">
  <li>Перший запис</li>
```

```
<li>Другий запис</li>
</ul>
```

У цій роботі частіше зручно використовувати таблицю, бо вона природно відображає «рядки записів».

Таблиця: стандартний спосіб показати записи

Таблиця в HTML має чітку структуру:

- `<table>` – контейнер таблиці.
- `<thead>` – «шапка» з назвами колонок.
- `<tbody>` – тіло таблиці (рядки даних).
- `<tr>` – рядок.
- `<th>` – заголовок колонки (у `thead`).
- `<td>` – комірка з даними (у `tbody`).

Шаблон таблиці, яку потім буде заповнювати JavaScript:

```
<table id="itemsTable">
  <thead>
    <tr>
      <th>#</th>
      <th>Назва</th>
      <th>Статус</th>
      <th>Дата</th>
      <th>Дії</th>
    </tr>
  </thead>

  <tbody id="itemsTableBody">
    <!-- рядки даних додає JS -->
  </tbody>
</table>
```

Важливий принцип: `thead` зазвичай статичний, а `tbody` динамічний.

Атрибути `id` і `class`: як «підчепити» елементи з CSS/JS

HTML-атрибути, без яких виконувати роботу буде незручно:

- `id` – унікальний ідентифікатор елемента (на сторінці має бути один такий `id`).
 - Зручно для JS: `document.getElementById("itemsTableBody")`
 - Зручно для CSS: `#itemsTable { ... }`
- `class` – клас стилю (може повторюватися для багатьох елементів).

- Зручно для CSS: `.field { ... }`
- Зручно для JS: `document.querySelectorAll(".delete-btn")`

Практичний приклад:

```
<div class="field">
  <label for="titleInput">Назва</label>
  <input id="titleInput" class="input" type="text" />
</div>
```

HTML-форми: елементи вводу, які використовуються в роботі

Форма в HTML – це стандартний механізм, який дозволяє користувачу ввести дані та «відправити» їх на обробку. У першій лабораторній роботі дані на сервер не відправляються, але форма використовується як зручний спосіб:

- зібрати значення з полів;
- виконати валідацію;
- додати запис у масив JavaScript;
- оновити таблицю/список на сторінці.

form як контейнер введення

Базовий вигляд:

```
<form id="createForm">
  <!-- поля вводу -->
  <button type="submit">Додати</button>
</form>
```

Ключові моменти:

- `id` потрібен, щоб JavaScript легко знаходив форму (`document.getElementById("createForm")`).
- Подія `submit` спрацьовує при натисканні кнопки `type="submit"` або `Enter` у полі.

У першій лабораторній роботі у JS обов'язково використовується `event.preventDefault()`, щоб сторінка не перезавантажувалась.

label і прив'язка до поля через for та id

Правильний шаблон для кожного поля:

```
<label for="userInput">Користувач</label>
<input id="userInput" type="text" />
```

Це важливо для:

- зручності користування (клік по напису переводить фокус у поле);
- читабельності коду;
- мінімальної доступності інтерфейсу.

input для тексту, чисел і дат

Текстове поле type="text"

Використовується для імені користувача, назви, теми, коду тощо.

```
<input id="titleInput" type="text" placeholder="Напр., Заявка на доступ" />
```

Практичні атрибути:

- required – поле обов’язкове.
- minlength, maxlength – контроль довжини.

```
<input id="userInput" type="text" required minlength="3" maxlength="30" />
```

Числове поле type="number"

Для «кількість місць», «пріоритет», «ліміт», «seats» тощо.

```
<input id="capacityInput" type="number" min="1" max="500" />
```

Важливо:

Браузер допомагає з форматом, але в JS все одно треба перевіряти, що значення введено і воно в діапазоні.

Дата і час type="date" та type="datetime-local"

Якщо у вашому варіанті є поле дати/часу:

```
<input id="dateInput" type="date" />
```

Або дата+час:

```
<input id="dateTimeInput" type="datetime-local" />
```

Зауваження:

- datetime-local дає «локальний» час без часового поясу. Для лабораторної роботи №1 це нормально.
- Часто зручніше ставити «CreatedAt» автоматично у JS, але якщо в завданні є «Date» як введення користувача – використовуйте поля вище.

select та option для полів з фіксованим набором значень

select потрібен для статусів, типів, рівнів важливості, категорій тощо.

```
<label for="statusSelect">Статус</label>
<select id="statusSelect" required>
  <option value="">Оберіть статус</option>
  <option value="New">New</option>
  <option value="InProgress">InProgress</option>
  <option value="Done">Done</option>
</select>
```

Правила хорошого тону для лабораторної роботи №1:

- Додайте перший «порожній» варіант value="", щоб було що перевіряти у валідації.
- Значення в value має бути коротким і стабільним (його зручно зберігати в масиві), а текст усередині option – те, що бачить користувач.

textarea для коментарів і описів

Коли потрібно ввести довший текст (опис, коментар, повідомлення):

```
<label for="commentInput">Коментар</label>
<textarea id="commentInput" rows="4"></textarea>
```

Практичні атрибути:

- maxlength – щоб не вводили «роман на 50 сторінок».
- required – якщо коментар обов'язковий.

Кнопки і їхні типи

У формах є принципова різниця між типами кнопок:

- type="submit" – запускає submit форми.
- type="button" – просто кнопка, не «відправляє» форму.

Приклад:

```
<div class="buttons">
  <button type="submit">Додати</button>
  <button type="button" id="resetBtn">Очистити</button>
</div>
```

Це критично, щоб кнопка «Очистити» випадково не запускала додавання запису.

Поля для повідомлень про помилки

У лабораторній роботі №1 бажано одразу закласти місця, куди JS буде ВИВОДИТИ ТЕКСТ ПОМИЛОК:

```
<div class="field">
  <label for="userInput">Користувач</label>
  <input id="userInput" type="text" />
  <p class="error-text" id="userError"></p>
</div>
```

Перевага такого підходу:

- помилки не «стрибають» по сторінці;
- структура зрозуміла: у кожного поля є свій блок для повідомлення.

Мінімальний шаблон форми у лабораторній роботі №1

Універсальна структура, яку можна адаптувати під будь-який варіант:

```
<form id="createForm">
  <div class="field">
    <label for="field1Input">Поле 1</label>
    <input id="field1Input" type="text" />
    <p class="error-text" id="field1Error"></p>
  </div>

  <div class="field">
    <label for="field2Select">Поле 2</label>
    <select id="field2Select">
      <option value="">Оберіть значення</option>
      <option value="A">A</option>
      <option value="B">B</option>
    </select>
    <p class="error-text" id="field2Error"></p>
  </div>

  <div class="field">
    <label for="field3Text">Коментар</label>
    <textarea id="field3Text" rows="4"></textarea>
    <p class="error-text" id="field3Error"></p>
  </div>

  <div class="buttons">
    <button type="submit">Додати</button>
    <button type="button" id="resetBtn">Очистити</button>
  </div>
</form>
```

CSS: базові принципи стилізації сторінки

CSS (Cascading Style Sheets) відповідає за те, як виглядає ваша сторінка: шрифти, відступи, рамки, вирівнювання, розкладка елементів. У лабораторній роботі №1 CSS не є самоціллю, але без базових стилів інтерфейс виходить нечитабельним, і студентам складніше тестувати логіку.

Головна ідея: HTML задає структуру, CSS – правила оформлення цієї структури.

Як підключається CSS і чому це важливо

CSS зазвичай підключається зовнішнім файлом:

```
<link rel="stylesheet" href="styles.css" />
```

Переваги:

- стилі не «змішуються» з HTML;
- простіше змінювати вигляд сторінки, не чіпаючи логіку або розмітку;
- у наступних лабораторних можна буде розширювати оформлення без переписування структури.

Каскадність і пріоритет правил (дуже коротко)

Назва CSS містить слово cascading («каскадність») – це означає, що на один елемент можуть діяти різні правила, і браузер обирає «сильніше» за пріоритетом.

На рівні лабораторної роботи №1 достатньо знати:

- правило за id зазвичай сильніше за правило за class;
- правило за class зазвичай сильніше за правило за тегом;
- якщо правила однакові за «силою», перемагає те, що написано нижче у файлі.

Селектори, які потрібні в лабораторній роботі №1

За тегом

Застосовує стиль до всіх елементів певного типу:

```
table { border-collapse: collapse; }
input { padding: 6px; }
```

За класом (найчастіший варіант)

Клас можна повторювати на багатьох елементах:

```
.field { margin-bottom: 12px; }
.error-text { font-size: 12px; }
```

HTML приклад:

```
<div class="field">
  ...
  <p class="error-text"></p>
</div>
```

За id

id унікальний. Зручно для великих блоків (таблиця, контейнер):

```
#itemsTable { width: 100%; }
#list-section { margin-top: 20px; }
```

Вкладені селектори (контекст)

Застосовує правило до елементів всередині іншого елемента:

```
.field input { width: 100%; }
table th { text-align: left; }
```

Це зручно, коли є бажання стилізувати деякі input на сторінці інакше.

Базові властивості, які роблять сторінку «читабельною»

Шрифти і текст

Зазвичай достатньо задати шрифт для body:

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
}
```

Для заголовків можна задати відступи:

```
h1 { margin-bottom: 8px; }
h2 { margin-top: 20px; }
```

Відступи і рамки: margin, padding, border

- margin – зовнішній відступ (простір навколо елемента).
- border – рамка.
- padding – внутрішній відступ (простір між контентом і рамкою).

Приклад «карточного» блоку секції:

```
section {
  margin-bottom: 20px;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
}
```

Розміри: width, max-width

Щоб сторінка не була «на всю ширину» на великих екранах, часто роблять контейнер:

```
.container {
  max-width: 900px;
  margin: 0 auto;
  padding: 16px;
}
```

HTML:

```
<div class="container">
  <!-- весь контент сторінки -->
</div>
```

Фоновий колір і контраст

Досить корисно візуально відділяти області:

```
body { background: #f5f5f5; }
section { background: #fff; }
```

Оформлення форми: «нормальний» вигляд полів

Типовий набір стилів для полів:

```
.field {
  margin-bottom: 12px;
}

.field label {
  display: block;
  margin-bottom: 4px;
}

.field input,
.field select,
.field textarea {
  width: 100%;
  padding: 8px;
  border: 1px solid #bbb;
  border-radius: 4px;
  box-sizing: border-box;
}
```

Ключовий момент тут – `box-sizing: border-box;`: тоді `width: 100%` буде враховувати рамку і `padding`, і поля не «вилазитимуть» за контейнер.

Таблиця: мінімальне оформлення для читабельності

Базовий варіант:

```
table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  border: 1px solid #ccc;
  padding: 8px;
}

th {
  background: #eee;
}
```

`border-collapse: collapse;` робить таблицю акуратнішою – рамки не подвоюються.

Стани валідації: стилі для помилок (обов'язково для «добре»)

У лабораторній роботі №1 зручно реалізувати помилки через CSS-класи, які JS додає/знімає.

Клас для невалідного поля

```
.invalid {
  border-color: #c00;
}
```

Використання: JS додає клас `invalid` на конкретний `input/select/textarea`.

Текст помилки

```
.error-text {
  margin: 4px 0 0;
  font-size: 12px;
  color: #c00;
  min-height: 14px; /* щоб верстка не "стрибала" при появі/зникненні тексту */
}
```

`min-height` корисний тим, що навіть якщо помилки немає, місце «зарезервоване», і форма не смикається.

Блок кнопок: елементарне оформлення

```
.buttons {
```

```

margin-top: 12px;
display: flex;
gap: 8px;
}

button {
padding: 8px 12px;
border: 1px solid #888;
border-radius: 4px;
cursor: pointer;
}

```

Навіть без «красивого дизайну» це дає акуратний інтерфейс.

Рекомендований мінімальний styles.css для лабораторної роботи №1

Як практичний орієнтир (можна вставляти як стартовий шаблон):

```

body {
font-family: Arial, sans-serif;
font-size: 16px;
background: #f5f5f5;
margin: 0;
}

.container {
max-width: 900px;
margin: 0 auto;
padding: 16px;
}

section {
background: #fff;
padding: 12px;
border: 1px solid #ccc;
border-radius: 6px;
margin-bottom: 16px;
}

.field {
margin-bottom: 12px;
}

.field label {
display: block;
margin-bottom: 4px;
}

.field input,
.field select,
.field textarea {
width: 100%;
padding: 8px;
border: 1px solid #bbb;
border-radius: 4px;
box-sizing: border-box;
}

```

```

}

.invalid {
  border-color: #c00;
}

.error-text {
  margin: 4px 0 0;
  font-size: 12px;
  color: #c00;
  min-height: 14px;
}

.buttons {
  margin-top: 12px;
  display: flex;
  gap: 8px;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  border: 1px solid #ccc;
  padding: 8px;
}

th {
  background: #eee;
}

```

CSS-розкладка: властивість display

Властивість `display` визначає, як елемент «поводиться» у потоці сторінки: чи займає він весь рядок, чи розміщується в одному рядку з іншими елементами, чи працює як контейнер для розкладки дочірніх елементів. Для лабораторної роботи №1 `display` потрібен насамперед, щоб акуратно розташувати поля форми, підписи, кнопки та (за потреби) групи елементів.

display: block

Блокові елементи:

- за замовчуванням починаються з нового рядка;
- займають доступну ширину контейнера (як правило, «розтягуються»);
- добре підходять для великих секцій інтерфейсу: `div`, `section`, `p`, `h1` тощо.

Приклад: зробити `label` блоковим, щоб підпис був над полем:


```
.field label {
  display: block;
  margin-bottom: 4px;
}
```

У результаті label і input йдуть один під одним, що найзручніше для форми.

display: inline

Інлайнні елементи:

- не починаються з нового рядка;
- займають ширину «по контенту»;
- ігнорують width і height (у більшості випадків);
- підходять для дрібних фрагментів тексту: span, strong, em.

Приклад: підпис у тексті:

```
<p>Статус: <span class="status">New</span></p>

.status {
  /* inline за замовчуванням */
  font-weight: bold;
}
```

display: inline-block

inline-block – компромісний варіант:

- елемент стоїть в одному рядку з іншими (як inline);
- але при цьому можна задавати width, height, padding, margin (як у block).

Практичний сенс у лабораторній роботі №1: інколи зручно робити «маленькі блоки в ряд» (але якщо рядків/кнопок стає багато – зазвичай краще використовувати flex).

Приклад: компактні «мітки»/бейджі:

```
.badge {
  display: inline-block;
  padding: 2px 6px;
  border: 1px solid #888;
  border-radius: 10px;
  font-size: 12px;
}
```

display: none

`display: none` повністю прибирає елемент із розкладки (він не займає місця). Це зручно, якщо треба приховувати/показувати елементи через JS (наприклад, блок «Немає записів»).

Приклад:

```
.hidden {
  display: none;
}
```

Тоді в JS можна додавати/знімати клас `hidden` для повідомлення або блоку.

display: flex (базово)

`flex` перетворює елемент на контейнер, який розкладає своїх дітей у рядок або стовпчик. Це найпростіший спосіб:

- розташувати кнопки в один ряд з відступами,
- зробити «панель керування» (пошук + фільтр + кнопка),
- вирівняти елементи по центру.

Типовий кейс лабораторної роботи №1: рядок кнопок.

```
.buttons {
  display: flex;
  gap: 8px;
}
```

За замовчуванням `flex` розміщує дочірні елементи в ряд (`flex-direction: row`). Якщо треба «в стовпчик»:

```
.form-layout {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

Корисні властивості для простих випадків:

- `gap` – відстань між елементами (зручніше, ніж виставляти `margin` кожному).
- `justify-content` – вирівнювання по головній осі (наприклад, «розсунути» елементи по краях).
- `align-items` – вирівнювання по поперечній осі (наприклад, вирівняти кнопки по центру по висоті).

Наприклад, якщо треба кнопку справа:

```
.toolbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

JavaScript у порівнянні з C++: мінімально необхідні відмінності

У лабораторній роботі №1 JavaScript використовується для «поведінки» сторінки: зчитати значення полів форми, перевірити їх, додати запис у масив, перемалювати таблицю, обробити натискання кнопок. Важливо швидко засвоїти кілька ключових відмінностей, щоб уникати типових помилок.

Динамічна типізація: немає int, double, string як у C++

У C++ тип змінної задається під час компіляції. У JavaScript тип значення визначається під час виконання.

У сучасному JS використовують:

- `const` – змінна, якій не можна перевизначити посилання (але об'єкт усередині може змінюватися),
- `let` – звичайна змінна, яку можна перевизначати,
- `var` – застарілий варіант (не використовується).

```
const maxItems = 100;
let count = 0;

count = count + 1;    // ok
// maxItems = 200;    // помилка: Assignment to constant variable
```

Практичне правило: все, що не має змінюватися – `const`, решта – `let`.

Базові типи даних, які реально зустрінете

Найчастіше в лабораторній роботі №1 треба працювати з:

- `string` – рядки,
- `number` – числа (і цілі, і дробові),
- `boolean` – `true/false`,
- `null` – «значення є, але воно порожнє»,
- `undefined` – «значення не задано»,
- `object` – об'єкти (включно з масивами),

- Array – масиви (це окремий різновид об'єктів).

Перевірка типу:

```
typeof "abc"      // "string"
typeof 123        // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof null       // "object" (історична "особливість" JS)
```

Зверніть увагу: значення з форми – це майже завжди рядки

Навіть якщо поле має type="number", .value повертає рядок.

```
const capacityStr = document.getElementById("capacityInput").value;
// capacityStr має тип string, наприклад "15"
```

Якщо потрібне число – треба явно перетворювати:

```
const capacity = Number(capacityStr); // або parseInt(capacityStr, 10)
```

Підводні камені:

- Number("") дає 0 (часто це небажано) – тому порожнє поле варто перевіряти окремо.
- Number("abc") дає NaN (Not-a-Number). Перевірка: Number.isNaN(value).

Об'єкти замість структур: запис сутності як «словник полів»

У C++ треба було б описали struct або class. У JS достатньо використовувати об'єкт:

```
const item = {
  id: 1,
  title: "Заявка на доступ",
  status: "New",
  comment: "Потрібен доступ на завтра",
  createdAt: "2026-01-10T12:30"
};
```

Читання/запис полів:

```
console.log(item.title);
item.status = "Done";
```

Масиви: основне сховище даних

У лабораторній роботі №1 дані зберігаються в масиві об'єктів:

```
const items = [];

console.log(items.length);           // кількість
```

Корисні методи, які покривають більшість потреб:

- `push(...)` – додає елемент в кінець масиву
- `pop(...)` – повертає останній елемент масиву, видаляючи його з масиву
- `find(...)` – знайти перший елемент
- `filter(...)` – створити новий масив за умовою (зручно для видалення)
- `map(...)` – перетворити елементи (часто для рендеру)
- `sort(...)` – сортування (для «відмінно»)

Приклади:

```
// знайти за id
const found = items.find(x => x.id === 10);

// видалити за id (створюємо новий масив)
const newItems = items.filter(x => x.id !== 10);
```

Важливо: `filter` не змінює масив «на місці», він повертає новий.

Порівняння: `===` замість `==`

У JS є два варіанти порівняння:

- `==` – може робити неочевидні автоматичні перетворення типів,
- `===` – строге порівняння (і значення, і тип).

Рекомендація: по замовчуванню використовувати тільки `===` і `!==`.

```
"5" == 5    // true   (небажано)
"5" === 5   // false  (правильно й передбачувано)
```

Області видимості: блокова, як у сучасному C++

`let` і `const` мають блокову область видимості (всередині `{ ... }`), що схоже на C++.

```
if (true) {
  const x = 10;
}
// x тут недоступний
```

`var` поводиться інакше (функціональна область видимості), через це породжує плутанину – тому його використання не рекомендується.

Функції та стрілочні функції (arrow functions)

Функції в JS можна передавати як значення. У лабораторній роботі №1 це потрібно для обробників подій і для find/filter.

Звичайна функція:

```
function validateForm() {
  return true;
}
```

Стрілочна функція (часто зручніша):

```
const validateForm = () => {
  return true;
};
```

Короткий запис для лямбда-умов у масивах:

```
items.filter(x => x.status === "New");
```

Рядки-шаблони: зручно для формування HTML або тексту

Замість конкатенації як у C++ ("a" + b + "c") часто використовують template literals:

```
const rowHtml = `<tr><td>${item.title}</td><td>${item.status}</td></tr>`;
```

Помилки та відладка: console.log і винятки

У браузері базовий інструмент відладки – консоль DevTools.

```
console.log(items);
console.log("title =", title);
```

Як і в C++, у JS є винятки:

```
throw new Error("Некоректні дані");
```

У лабораторній роботі №1 зазвичай достатньо не кидати винятки, а показувати помилки у UI. Але корисно знати, що «падіння скрипта» часто видно в консолі як помилка.

Підключення JavaScript та життєвий цикл сторінки

У лабораторній роботі JavaScript працює з DOM-елементами (форма, поля вводу, таблиця), тому важливо розуміти коли саме ці елементи стають доступними для коду. Більшість типових помилок на старті («не знаходить

елемент», «Cannot read properties of null») пов'язані саме з тим, що скрипт виконується раніше, ніж браузер встиг побудувати DOM.

Де зберігати код і як підключати файл app.js

Код доцільно тримати в окремому файлі, наприклад app.js, і підключати його в index.html:

```
<script src="app.js" defer></script>
```

Переваги:

- HTML не засмічується логікою;
- код легше читати та перевіряти;
- у наступних лабораторних простіше розширювати проект.

Що відбувається під час завантаження сторінки (спрощено)

Під час відкриття сторінки браузер робить таке:

- Завантажує HTML.
- «Розбирає» HTML і будує DOM (дерево елементів сторінки).
- Завантажує підключені ресурси (CSS, JS, картинки).
- Виконує JavaScript (залежно від того, як він підключений).
- Відображає сторінку та реагує на дії користувача.

Критично: код, який шукає елементи через document.getElementById(...) або querySelector(...), має запускатися після того, як DOM уже існує.

Чому рекомендовано defer

Якщо підключити скрипт так:

```
<script src="app.js"></script>
```

то браузер може виконати app.js ще до того, як DOM буде повністю побудований. Тоді цей код:

```
const form = document.getElementById("createForm");
form.addEventListener("submit", ...);
```

може «впасти», бо form буде null.

Атрибут defer вирішує це:

```
<script src="app.js" defer></script>
```

Що це означає:

- скрипт завантажиться,
- але виконається лише після того, як HTML розібраний і DOM сформований,
- отже, елементи форми/таблиці вже існують у момент виконання коду.

Практичний висновок: якщо використовується `defer`, то можна безпечно писати ініціалізацію одразу в «верхній» частині `app.js`.

Альтернатива: підключення `<script>` в кінці `body`

Інший поширений підхід:

```
<body>
  <!-- контент сторінки -->
  <script src="app.js"></script>
</body>
```

Тут логіка така сама: HTML уже завантажений, тому DOM існує, і скрипт може працювати з елементами.

У роботі допустимі обидва варіанти, але для єдиного стилю та передбачуваності рекомендовано:

`<script ... defer>` у `<head>`.

Подія `DOMContentLoaded`: коли вона може знадобитися

Іноді можна зустріти код такого вигляду:

```
document.addEventListener("DOMContentLoaded", () => {
  // ініціалізація
});
```

Це означає: «виконати код тоді, коли DOM побудований».

У даній роботі `DOMContentLoaded` може бути корисним, якщо:

- скрипт підключений без `defer` і не в кінці `body`;
- або є бажання навмисно хочете відкласти ініціалізацію.

Якщо у скрипт підключено з `defer`, то зазвичай `DOMContentLoaded` не потрібен (код і так запуститься в правильний момент). Але знати про нього корисно, бо це типовий «страхувальний механізм».

«Точка входу» app.js

Зручно мислити так: при старті сторінки виконується ініціалізація:

1. Знаходяться потрібні DOM-елементи (форма, поля, tbody таблиці, кнопки).
2. Виконується підпис на події (submit, click).
3. Якщо використовується localStorage – дані відновлюються з нього.
4. Викликається render() для первинного відображення.

Схематично:

```
// 1) знайти елементи  
// 2) addEventListener(...)  
// 3) loadFromStorage()  
// 4) render()
```

Це «скелет» застосунку, який збережеться і в наступних лабораторних: змінюватиметься тільки наповнення (більше сутностей, запити на бекенд, безпека тощо).

Типові помилки й як їх розпізнавати

1. Cannot read properties of null
 - 1.1. Причина: document.getElementById(...) повернув null, бо елемент ще не існував у DOM або id написаний інакше.
 - 1.2. Як перевірити:
 - 1.2.1. відкрийте DevTools → Console;
 - 1.2.2. перевірте, чи дійсно в HTML є елемент з таким id;
 - 1.2.3. переконайтесь, що скрипт підключений з defer або в кінці body.
2. Скрипт не виконується взагалі
 - 2.1. Причина: неправильний шлях до файлу або помилка синтаксису.
 - 2.2. Як перевірити:
 - 2.2.1. DevTools → Network (чи завантажився app.js);
 - 2.2.2. Console (чи є помилки);
 - 2.2.3. додайте тимчасово console.log("app loaded"); на початку app.js.

DOM: пошук елементів і робота з елементами сторінки

DOM (Document Object Model) – це «дерево» об'єктів, яке браузер будує з вашого HTML. JavaScript працює не з текстом HTML напряду, а з DOM-елементами: знаходить їх, читає значення, змінює вміст, додає/видаляє вузли, вішає обробники подій.

У даному курсі DOM буде потрібен для трьох практичних задач:

- зчитати дані з форми;
- показати/оновити таблицю або список;
- реагувати на натискання кнопок.

Як знаходити елементи на сторінці

`document.getElementById(...)`

Найпростіший і швидкий спосіб, при роботі з унікальними елементами:

```
const form = document.getElementById("createForm");
const tableBody = document.getElementById("itemsTableBody");
```

Використовуйте, коли в HTML точно поставлений id.

`document.querySelector(...)` і `querySelectorAll(...)`

Ці методи приймають CSS-селектор.

```
const firstError = document.querySelector(".error-text"); // перший елемент з класом
const allFields = document.querySelectorAll(".field input"); // NodeList усіх input у .field
```

Практичне правило:

- для одиничних важливих елементів зручніше `getElementById`;
- для груп елементів – `querySelectorAll`.

Як читати дані з форми

Для полів введення (`input/select/textarea`) основне джерело даних – властивість `.value`.

```
const userName = document.getElementById("userInput").value;
const status = document.getElementById("statusSelect").value;
const comment = document.getElementById("commentInput").value;
```

Типовий нюанс: `.value` повертає рядок. Якщо потрібно число:

```
const capacity = Number(document.getElementById("capacityInput").value);
```

Щоб «прибрати» випадкові пробіли на початку/в кінці:

```
const title = document.getElementById("titleInput").value.trim();
```

Як змінювати елементи: класи, атрибути, видимість

Зміна CSS-класів: *classList*

Це основа для підсвічування помилок:

```
const el = document.getElementById("userInput");
el.classList.add("invalid");
el.classList.remove("invalid");
```

Зміна атрибутів

Наприклад, інколи треба вимкнути кнопку:

```
document.getElementById("saveBtn").setAttribute("disabled", "disabled");
// або простіше:
document.getElementById("saveBtn").disabled = true;
```

Приховування/показ: клас *.hidden*

Якщо є CSS:

```
.hidden { display: none; }
```

то в JS:

```
const emptyState = document.getElementById("emptyState");
emptyState.classList.toggle("hidden", items.length > 0);
```

***innerHTML* як основний спосіб рендеру у лабораторній роботі**

innerHTML дозволяє вставляти у елемент готову HTML-розмітку рядком.

Для початку це зручно, бо:

- швидко формувати таблицю/список;
- легко «перемалювати» весь *tbody* однією операцією.

Приклад: рендер таблиці з масиву *items*:

```
function renderTable(items) {
  const tbody = document.getElementById("itemsTableBody");

  const rowsHtml = items.map((item, index) => `
    <tr>
      <td>${index + 1}</td>
      <td>${item.title}</td>
      <td>${item.status}</td>
      <td>${item.createdAt}</td>
    </td>
```

```

        <button type="button" class="delete-btn" data-
id="${item.id}">Видалити</button>
      </td>
    </tr>
  `).join("");
  tbody.innerHTML = rowsHtml;
}

```

Ключові моменти:

- HTML для кожного рядка будується через template literals `...`;
- `map(...).join("")` дає один великий рядок;
- присвоєння `tbody.innerHTML = ...` замінює весь вміст `tbody`.

У наступних лабораторних це стане точкою, де студенти будуть покращувати безпеку рендеру. Але зараз застосовується `innerHTML` як простий інструмент.

Як додати кнопки «Видалити/Редагувати» і прив'язати їх до запису

Оскільки рядки таблиці створюються динамічно, зручно «прив'язувати» кнопку до елемента через `data-*` атрибут.

У HTML-рядку:

```
<button type="button" class="delete-btn" data-id="123">Видалити</button>
```

У JS це читається як:

```
const id = Number(button.dataset.id);
```

Типові помилки і як їх діагностувати

1. `document.getElementById(...)` повертає `null`
 - 1.1. Причина: елемент не існує або `id` не збігається (регістр, помилка в назві).
 - 1.2. Рішення: перевірити HTML і назву `id`, переконатися, що `app.js` підключено з `defer`.
2. Після `tbody.innerHTML = ...` «зникають» обробники на кнопках
 - 2.1. Причина: перевставили HTML – старі кнопки замінилися новими.
 - 2.2. Рішення: або використовувати делегування, або після кожного рендеру повторно підписуватись на кнопки.

3. У таблиці «ламається» верстка, якщо коментар містить спецсимволи/теги
 - 3.1. Причина: innerHTML інтерпретує вставлений текст як HTML.
 - 3.2. У лабораторній роботі №1 це допустимо як навчальний крок; у наступних роботах це буде виправлено.

Події: як обробляти введення і натискання

Подія (event) – це сигнал від браузера, що «щось сталося»: користувач натиснув кнопку, відправив форму, змінив значення поля, навів курсор тощо. Наразі інтерес представляють, передусім, події, що потрібні для роботи з формою та таблицею:

- submit – коли користувач відправляє форму (натискає «Додати» або Enter).
- click – коли натискає кнопки (наприклад, «Видалити», «Редагувати», «Очистити»).
- (опційно) input або change – якщо треба валідувати «на льоту», але це не обов’язково.

Як підписатися на подію: addEventListener

Базовий спосіб обробки подій у JS:

```
element.addEventListener("eventName", handlerFunction);
```

- де eventName – назва події ("submit", "click"...);
- handlerFunction – функція-обробник, яка виконається при настанні події.

Приклад:

```
const btn = document.getElementById("resetBtn");
btn.addEventListener("click", () => {
  console.log("Натиснули кнопку Очистити");
});
```

Подія submit: правильна обробка форми

Форма за замовчуванням намагається «відправити» дані та перезавантажити сторінку. Наразі це не потрібно, тому в обробнику submit обов’язково робимо:

event.preventDefault() – скасувати стандартну поведінку браузера.

Шаблон:

```
const form = document.getElementById("createForm");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  // 1) зчитати поля
  // 2) перевірити дані
  // 3) додати/оновити запис у масиві
  // 4) оновити таблицю (render)
});
```

Практичний сенс: після натискання «Додати» сторінка не перезавантажується, а таблиця оновлюється динамічно.

Типовий «найплайн» обробки: *read* → *validate* → *update* → *render*

Щоб код не перетворився на суцільний блок, зручно дотримуватися одного шаблону:

1. read – зчитали дані з форми (.value)
2. validate – перевірили і показали помилки
3. update – оновили дані в масиві (push, filter, тощо)
4. render – перемалювали таблицю/список (innerHTML = ...)

Наприклад, логіка в обробнику submit виглядає так:

```
form.addEventListener("submit", (event) => {
  event.preventDefault();

  const dto = readForm();
  const isValid = validate(dto);

  if (!isValid) return;

  addItem(dto);
  renderTable(items);
  resetForm();
});
```

Це не «вимога мови», а практична структура, яка полегшує перевірку і розвиток проекту.

Подія click: кнопки «Очистити», «Видалити», «Редагувати»**Статичні кнопки**

Якщо кнопка існує в HTML і не перегенерується через innerHTML, можна підписатися напямую:

```
document.getElementById("resetBtn").addEventListener("click", () => {
  resetForm();
  clearErrors();
});
```

Динамічні кнопки в таблиці

У лабораторній роботі №1 таблиця генерується через innerHTML. Це означає, що кнопки «Видалити/Редагувати» створюються заново після кожного render().

Є два підходи:

1. простий, але менш зручний: після кожного render() знову знайти всі кнопки і повісити на них обробники.
2. рекомендований: делегування подій – один обробник на контейнер (tbody) ловить усі кліки всередині.

Делегування подій для таблиці

Приклад: у рендері створюються кнопки з data-id:

```
<button type="button" class="delete-btn" data-id="123">Видалити</button>
<button type="button" class="edit-btn" data-id="123">Редагувати</button>
```

А один обробник на tbody визначає, по якій кнопці клікнули:

```
const tbody = document.getElementById("itemsTableBody");

tbody.addEventListener("click", (event) => {
  const target = event.target;

  if (target.classList.contains("delete-btn")) {
    const id = Number(target.dataset.id);
    deleteItemById(id);
    renderTable(items);
    return;
  }

  if (target.classList.contains("edit-btn")) {
    const id = Number(target.dataset.id);
    startEdit(id); // підставити дані в форму
    return;
  }
});
```

Переваги:

- не потрібно перевішувати обробники після кожного innerHTML;
- код коротший і стабільніший;

- легше додавати нові кнопки (наприклад, «Деталі»).

Об'єкт події event

У обробнику події найчастіше використовують:

- `event.preventDefault()` – для форми.
- `event.target` – елемент, по якому клікнули (або який ініціював подію).

Для делегування `event.target` – ключовий:

```
const target = event.target;
```

І далі перевіряється клас або читається dataset.

Події input / change для «валідації на льоту»

Це не обов'язково для даної лабораторної роботи, але корисно знати:

- `input` – спрацьовує під час введення (кожна зміна).
- `change` – спрацьовує, коли поле втратило фокус після зміни.

Приклад: очищати помилку одразу, коли користувач почав виправляти

поле:

```
document.getElementById("userInput").addEventListener("input", () => {
  clearFieldError("userInput", "userError");
});
```

Якщо не хочете ускладнювати – робіть валідацію лише на submit.

Типові помилки при роботі з подіями

1. Форма перезавантажується
 - 1.1. Причина: забули `event.preventDefault()` в обробнику події submit.
 - 1.2. Рішення: додати `event.preventDefault()` в обробник події submit.
2. Кнопка «Очистити» відправляє форму
 - 2.1. Причина: у HTML кнопка без `type="button"` (за замовчуванням button у формі може поводитися як submit).
 - 2.2. Рішення: для не-submit кнопок завжди ставити `type="button"`.
3. Кнопки в таблиці не реагують після рендеру
 - 3.1. Причина: перевставили `innerHTML` і загубили підписки на кнопках.
 - 3.2. Рішення: делегування подій на контейнер.

Валідація введення на клієнті

Валідація – це перевірка введених користувачем даних до того, як вони будуть «прийняті» (додані в масив, відображені в таблиці тощо). У першій лабораторній роботі валідація потрібна з двох причин:

1. Якість даних: не додавати порожні або некоректні записи.
2. Зручність користувача: показати, що саме введено неправильно, і як це виправити.

В цій роботі робиться мінімально достатня валідація: обов'язковість полів, довжина рядків, діапазони чисел/дат, «обрано зі списку». Без складних регулярних виразів і без серверної частини.

HTML-валідація vs JS-валідація (і чому потрібні обидві)

HTML-валідація

Це атрибути на полях:

- required
- minlength, maxlength
- min, max

Наприклад:

```
<input id="userInput" type="text" required minlength="3" maxlength="30" />
```

Плюс: браузер «сам» підкаже, що поле порожнє або надто коротке.

Мінуси:

- повідомлення браузера часто «не у вашому стилі»;
- складно робити однакову логіку для різних полів;
- не завжди можна коректно пояснити помилку.

JS-валідація

Це власний код, який:

- перевіряє дані,
- підсвічує поля,
- виводить текст помилки в потрібному місці.

JS-валідація використовується як основний контроль, а HTML-атрибути – як додаткова «страховка».

Загальна схема валідації

Зручно робити валідацію в окремій функції, яка:

1. приймає дані форми (об'єкт),
2. повертає true/false,
3. і паралельно показує помилки в UI.

Шаблон:

```
function validate(dto) {
  clearErrors();

  let isValid = true;

  // перевірки...
  // якщо помилка:
  // showError(...); isValid = false;

  return isValid;
}
```

Важливо: перед новою перевіркою очищають старі помилки, щоб форма не «залипала» у невалідному стані.

Як показувати помилки: клас + текст

Найпрактичніший підхід у першій роботі:

1. на поле додаємо клас `.invalid`;
2. у `p.error-text` записуємо повідомлення.

Припустимо, HTML виглядає наступним чином:

```
<input id="userInput" type="text" />
<p class="error-text" id="userError"></p>
```

CSS виглядає наступним чином:

```
.invalid { border-color: #c00; }
.error-text { color: #c00; font-size: 12px; }
```

Тоді функція для показу помилки:

```
function showError(inputId, errorId, message) {
  document.getElementById(inputId).classList.add("invalid");
  document.getElementById(errorId).innerHTML = message;
}
```

І функція очищення:

```
function clearError(inputId, errorId) {
  document.getElementById(inputId).classList.remove("invalid");
  document.getElementById(errorId).innerHTML = "";
}
```

Очищення всіх помилок (перед валідацією)

Найпростіше – явно перерахувати поля, які перевіряються:

```
function clearErrors() {
  clearError("userInput", "userError");
  clearError("statusSelect", "statusError");
  clearError("commentInput", "commentError");
}
```

Це трохи багатослівно, але для початку це нормально: код прозорий і легко перевіряється.

Типові перевірки для лабораторної роботи №1 (приклади)

Перевірка на порожнє значення

```
const user = dto.userName.trim();
if (user === "") {
  showError("userInput", "userError", "Поле є обов'язковим.");
  isValid = false;
}
```

Перевірка довжини рядка

```
if (user.length < 3 || user.length > 30) {
  showError("userInput", "userError", "Довжина має бути від 3 до 30 символів.");
  isValid = false;
}
```

Перевірка select (обрано чи ні)

Якщо перший option має value="", то перевірка проста:

```
if (dto.status === "") {
  showError("statusSelect", "statusError", "Оберіть значення зі списку.");
  isValid = false;
}
```

Перевірка числа (діапазон)

```
if (dto.capacityStr.trim() === "") {
  showError("capacityInput", "capacityError", "Вкажіть число.");
  isValid = false;
} else {
  const capacity = Number(dto.capacityStr);
  if (Number.isNaN(capacity) || capacity < 1 || capacity > 500) {
```

```

    showError("capacityInput", "capacityError", "Число має бути в діапазоні 1-500.");
    isValid = false;
  }
}

```

Перевірка дат (якщо є DateFrom / DateTo)

У спрощеному вигляді:

```

if (dto.dateFrom === "" || dto.dateTo === "") {
  showError("dateFromInput", "dateFromError", "Вкажіть дату.");
  isValid = false;
} else if (dto.dateFrom > dto.dateTo) {
  showError("dateToInput", "dateToError", "Дата завершення не може бути раніше дати початку.");
  isValid = false;
}

```

Для type="date" значення мають формат YYYY-MM-DD, і їх можна порівнювати як рядки (це працює коректно для такого формату). Для datetime-local краще перетворювати на Date, але в поки можна залишити простіший підхід, якщо не ускладнювати.

Де тримати «дані форми»: DTO-об'єкт

Зручно зчитувати значення в окремий об'єкт і вже його валідовати:

```

function readForm() {
  return {
    userName: document.getElementById("userInput").value,
    status: document.getElementById("statusSelect").value,
    comment: document.getElementById("commentInput").value
  };
}

```

Перевага:

- validate(dto) не залежить від DOM напряму (легше читати й тестувати логіку);
- у наступних лабораторних DTO-концепція стане основою для контрактів із бекендом.

Рекомендований мінімальний шаблон validate(dto)

```

function validate(dto) {
  clearErrors();
  let isValid = true;

  const user = dto.userName.trim();
  if (user === "") {

```

```

    showError("userInput", "userError", "Поле є обов'язковим.");
    isValid = false;
  } else if (user.length < 3 || user.length > 30) {
    showError("userInput", "userError", "Довжина має бути від 3 до 30
символів.");
    isValid = false;
  }

  if (dto.status === "") {
    showError("statusSelect", "statusError", "Оберіть значення зі списку.");
    isValid = false;
  }

  const comment = dto.comment.trim();
  if (comment.length < 5) {
    showError("commentInput", "commentError", "Коментар має містити щонайменше
5 символів.");
    isValid = false;
  }

  return isValid;
}

```

localStorage: що це і як мінімально користуватись

Дані за замовчуванням зберігаються в пам'яті сторінки (у масиві JavaScript). Це означає: якщо перезавантажити сторінку, усі додані записи зникнуть. Щоб зробити збереження між перезавантаженнями, використовують localStorage.

Що таке localStorage

localStorage – це вбудоване сховище браузера у форматі «ключ → значення», де:

- ключ – рядок,
- значення – рядок,
- дані зберігаються між перезавантаженнями і навіть після закриття браузера (поки користувач їх не очистить).

Важливо:

- localStorage прив'язаний до домену (у навчальних умовах це може бути file:// або локальний сервер).
- Якщо користувач «очистить дані сайту» – localStorage теж очиститься.

Основні операції

Поки достатньо трьох методів:

Запис:

```
localStorage.setItem("ключ", "значення");
```

Читання:

```
const value = localStorage.getItem("ключ"); // повертає рядок або null
```

Видалення:

```
localStorage.removeItem("ключ");
```

Очистити все для домену:

```
localStorage.clear();
```

Проблема: localStorage зберігає лише рядки

Ваші дані – це масив об'єктів `items`. Його не можна зберігати напряму, тому використовують JSON:

Перетворення в рядок:

```
const json = JSON.stringify(items);
```

Відновлення з рядка:

```
const items = JSON.parse(json);
```

Типовий сценарій: зберігати items після кожної зміни

Функція збереження

```
const STORAGE_KEY = "lr1_items";

function saveToStorage(items) {
  const json = JSON.stringify(items);
  localStorage.setItem(STORAGE_KEY, json);
}
```

Виклик: після `add/delete/edit` (тобто після будь-якої зміни даних):

```
addItem(dto);
saveToStorage(items);
renderTable(items);
```

Функція завантаження під час старту сторінки

```
function loadFromStorage() {
  const json = localStorage.getItem(STORAGE_KEY);
  if (json === null) return [];
}
```

```
try {
  const data = JSON.parse(json);
  return Array.isArray(data) ? data : [];
} catch {
  return [];
}
}
```

А на старті (у вашій «точці входу»):

```
let items = loadFromStorage();
renderTable(items);
```

try/catch потрібен на випадок, якщо в сховищі лежить пошкоджений або не-JSON рядок.

Де зберігати nextId (якщо використовується лічильник)

Якщо id генерується лічильником nextId, то після перезавантаження його теж треба відновити. Є два прості підходи:

Підхід А (рекомендований): не зберігати nextId, а після завантаження обчислити:

```
function computeNextId(items) {
  if (items.length === 0) return 1;
  const maxId = Math.max(...items.map(x => x.id));
  return maxId + 1;
}
```

Підхід В: зберігати nextId окремим ключем:

```
localStorage.setItem("lr1_nextId", String(nextId));
const nextId = Number(localStorage.getItem("lr1_nextId") ?? "1");
```

Коли саме викликати збереження

Щоб не забувати, тримайте правило: після кожної операції, що змінює масив items → saveToStorage(items) → render().

Типові місця:

- після додавання запису;
- після видалення;
- після редагування;
- (опційно) після «очистити всі записи».

Типові помилки і як їх уникнути

1. Зберегли об'єкт напряму (без JSON)
 - 1.1. `localStorage.setItem(key, items)` перетворить масив в рядок на кшталт "[object Object]".
 - 1.2. Рішення: завжди `JSON.stringify`.
2. `JSON.parse` «падає»
 - 2.1. Причина: у сховищі не JSON або дані пошкоджені.
 - 2.2. Рішення: `try/catch` і дефолт `[]`.
3. Дані відображаються, але після редагування/видалення не зберігаються
 - 3.1. Причина: забули викликати `saveToStorage` після цієї операції.
 - 3.2. Рішення: застосувати правило «будь-яка зміна → save → render».

Практична частина

Загальна постановка (наскрізний проект на семестр)

Протягом семестру студенти розробляють невеликий веб-застосунок за обраним варіантом (див. перелік варіантів нижче). Кожна наступна лабораторна робота – це новий інкремент: студенти не створюють новий проект «з нуля», а розширюють вже зроблене.

У лабораторній роботі №1 реалізується лише фронтенд без бекенда: дані зберігаються в пам'яті сторінки.

Що саме треба зробити в рамках лабораторної роботи №1?

На лабораторній роботі №1 необхідно реалізувати CRUD-мінімум для однієї сутності. Решта сутностей варіанту поки що «зарезервовані» під наступні роботи.

Обов'язково для лабораторної роботи №1: «Ключова сутність»

Обрати одну ключову сутність зі свого варіанта (рекомендація – див. Таблиця 1.2) і реалізувати для неї:

- HTML-форму введення,

- масив JavaScript для зберігання записів,
- виведення записів у таблицю/список,
- мінімальну валідацію,
- обробку подій (submit/click) та перерендер.

Порядок виконання роботи

1. Ознайомитись з теоретичним матеріалом.
2. Виконати індивідуальні завдання.
3. Створити публічний репозиторій на github/gitlab/інший провайдер, куди завантажити код. Остаточну версію позначити тегом «0.1.0».
4. Підготувати звіт по виконаній роботі, оформлений згідно з ДСТУ 3008:2015. У звіті вказати посилання на репозиторій з п.3.
5. Також у звіті необхідно вказати, що саме було реалізовано, які поля валідуються яким чином, прикріпити скріншоти успішної і неуспішної валідації введених даних.

Вимоги за рівнями оцінювання

Виконання «на задовільно»

1. Створити сторінку застосунку (заголовок з назвою варіанта).
2. Реалізувати форму додавання запису ключової сутності (4–5 полів, як вказано в Таблиця 1.1).
 - 2.1. у формі є label для кожного поля;
 - 2.2. кнопки мають type="submit"/type="button" де потрібно.
3. Зберігати записи у масиві JavaScript (in-memory).
4. Відображати записи в таблиці/списку після додавання без перезавантаження сторінки.
 - 4.1. у таблиці є thead/tbody і th;
5. CSS-мінімум:
 - 5.1. базова розкладка через Flexbox (наприклад, «форма зліва, список справа»),

5.2. box-sizing: border-box, нормальні відступи, читабельність.

6. Мінімальна валідація:

- 6.1. обов'язкові поля не можуть бути порожніми;
- 6.2. для Url/Email/Date/Number (якщо є) – базова перевірка формату/діапазону;
- 6.3. показати користувачу повідомлення про помилку (під формою або біля полів).
- 6.4. кнопка сабміту блокується або сабміт не проходить з явним поясненням

Виконання «на добре»

Усе з рівня «задовільно», а також:

- 1. Видалення запису зі списку (кнопка «Видалити» в рядку).
- 2. Якісніша валідація та UX:
 - 2.1. підсвічування невалідних полів (CSS-клас);
 - 2.2. помилки виводити адресно біля конкретного поля;
 - 2.3. очищення форми після успішного додавання.
- 3. Структура коду:
 - 3.1. винести логіку в окремі функції (наприклад: readForm(), validate(), addItem(), render()),
 - 3.2. кожному запису присвоювати id.
 - 3.3. мінімальний поділ коду: state (масив + поточні фільтри/форма), render, handlers

Виконання «на відмінно»

Усе з рівня «добре», а також:

- 1. Редагування запису:
 - 1.1. кнопка «Редагувати» підставляє значення в форму;
 - 1.2. кнопка «Зберегти» оновлює існуючий запис за id.
- 2. Пошук/фільтрація/сортування (реалізувати щонайменше 2 можливості):
 - 2.1. пошук за одним текстовим полем (наприклад, Title, UserName, Subject);

- 2.2. фільтр за select-полем (Status, Severity, Type, тощо);
- 2.3. сортування за датою/часом або за пріоритетом/важливістю.
- 3. Збереження між перезавантаженнями:
 - 3.1. зберігати масив у localStorage;
 - 3.2. при відкритті сторінки – відновлювати та рендерити.
- 4. делегування подій для динамічних списків

Додаткове задання (за бонусні бали)

- 1. Фреймворк (Angular/React/Vue): реалізувати функціонал лабораторної роботи на фреймворку (але базова версія без фреймворку теж має бути виконана).

Формальні критерії оцінювання

Задовільно

- 1. Є форма вводу + збереження даних у state (масив/об'єкти) + відображення списком/таблицею через функцію render().
- 2. Події обробляються коректно: мінімум submit для форми + події для дій зі списком (клік по кнопці/рядку).
- 3. Є клієнтська валідація (required/довжина/формат) з явним повідомленням користувачу, в чому помилка.
- 4. Семантичний HTML: label прив'язаний до кожного поля, у таблиці є thead/tbody і заголовки th, кнопки мають коректні type.
- 5. CSS-мінімум: розкладка на Flexbox (або Grid) + адекватні відступи/читабельність (не «все злиплось»).

Добре

- 1. Реалізовано **видалення** елементів зі списку + коректний перерендер.
- 2. Дані мають **стабільний ідентифікатор** (не індекс у масиві як «id»).
- 3. Код розбитий мінімум на логічні блоки: state, render, handlers (або 2 файли), без «все в одному скрипті».

- Додано базовий UX: очищення/фокус форми після успіху, блокування повторного сабміту або захист від дублювання.

Відмінно

- Реалізовано **редагування** (через форму/модальне/inline) з оновленням state.
- Делегування подій** для елементів списку/таблиці (без навішування обробників на кожну кнопку при кожному рендері).
- Фільтр** або **пошук** по списку (хоч один варіант) + коректна робота з state.
- Сортування** (мінімум за одним полем) без поломки редагування/видалення.
- Персистентність**: localStorage (load/save) або інший дозволений спосіб збереження стану.

Варіанти індивідуальних завдань

Таблиця 1.1 – Сервіс і обов’язкові сутності за варіантами

№	Сервіс	Сутності
1	Менеджер заявок на доступ до лабораторії	Users, AccessRequests (Date, Comments), Approvals
2	Трекер інцидентів кібербезпеки	Users, Incidents (Date, Tags, Comments)
3	Реєстр обладнання кафедри з заявками на видачу	InventoryItems (Number/Code), CheckoutRequests, Users
4	Дошка оголошень групи	Users, Posts, Comments
5	Каталог навчальних ресурсів з рейтингом	Resources, Ratings, Comments, Users
6	Сервіс «репорт вразливості» для навчальних проектів	Users, Reports (Severity, Status)
7	Сервіс реєстрації на факультатив/подію	Events, Registrations, Users
8	Система пропусків у комп’ютерний клас	Users, Passes, Reasons, Logs
9	«Заявка в техпідтримку» для студентів	Tickets, TicketMessages, Users, Statuses
10	Кабінет студента: перелік дисциплін та нотатки	Courses, Enrollments, PersonalNotes
11	Сервіс чергувань у лабораторії	Schedule, Shifts, Users, SwapRequests
12	Реєстр програмного забезпечення для лабораторій	SoftwareProducts, Licenses, Requests, Users
13	Система «опитування групи» (внутрішня)	Polls, Questions, Answers, Users
14	Сервіс «пул ідей для дипломів»	Ideas, Votes, Comments, Users
15	Сервіс «інвентаризація флешок/носіїв»	MediaDevices, Checkouts, Users, AuditLogs

Таблиця 1.2 – Рекомендовані поля ключових сутностей

№	Рекомендована ключова сутність	Рекомендовані поля ключової сутності
1	AccessRequests	UserName, Date(input date/datetime), AccessType(select), Comments(textarea), Status(select, по замовчуванню «Pending»)
2	Incidents	Date, Tag(select або text), Severity(select), Comments(textarea), Reporter(text)
3	CheckoutRequests	ItemCode, UserName, DateFrom, DateTo, Comment(textarea), Status(select, по замовчуванню «New»)
4	Posts	Title, Category(select), Body(textarea), Author(text), CreatedAt(auto)
5	Resources	Title, Url, Type(select), Description(textarea), Author(text або auto)
6	Reports	Title, Severity(select), Status(select, по замовчуванню «Open»), Description(textarea), Reporter
7	Events	Title, Date, Location, Capacity(number), Description(textarea)
8	Passes	UserName, Reason(select), ValidDate(date), Comment(textarea), Issuer(text/auto)
9	Tickets	Subject, Status(select), Priority(select), Message(textarea), Author
10	PersonalNotes	Course(select), Title, Note(textarea), CreatedAt(auto)
11	Shifts	Date, TimeSlot(select), UserName, Comment(textarea), Status(select)
12	SoftwareProducts	Name, Version, LicenseType(select), Seats(number), Comment(textarea)
13	Polls	Title, EndDate, Visibility(select), Description(textarea), Author
14	Ideas	Title, Category(select), Description(textarea), Author, Link(optional)
15	MediaDevices	DeviceCode/Serial, Type(select), Owner/UserName, Comment(textarea), CheckedOut(select true/false або Status)

Контрольні запитання

1. Яка роль файлів index.html, styles.css, app.js у вашому проєкті? Що саме має бути в кожному з них?
2. Навіщо при підключенні app.js часто використовують атрибут defer? Яка типова помилка виникає, якщо його не врахувати?
3. Чим відрізняються id і class в HTML? Наведіть приклад, де у вашій роботі доречно використати id, а де – class.

4. Які теги використовуються для відображення таблиці (або списку) даних? Для чого потрібні `thead` і `tbody` (або `ul/li`)?
5. Яка різниця між `button type="submit"` і `button type="button"` у формі? Яку помилку можна отримати, якщо тип не вказувати явно?
6. Які існують способи знаходження елементів у DOM? У чому їхня різниця?
7. Як зчитати значення з `input`, `select` та `textarea` через DOM? Яка властивість зазвичай використовується?
8. Чому значення з форми в JavaScript часто приходять як рядки навіть для `input type="number"`? Як правильно перетворити введені значення на число?
9. Що таке подія `submit`? Чому у вашому обробнику потрібен `event.preventDefault()`?
10. Що таке «валідація на клієнті» і для чого вона потрібна?
11. Як зазвичай показують помилку користувачу?
12. Що означає правило «змінив дані → викликав `render()`»? Чому зручно мати окрему функцію рендерингу?
13. Як саме формуються рядки таблиці/елементи списку в JS і вставляються у DOM? Яку роль тут відіграє `innerHTML`?
14. Як реалізується обробка кліків по кнопках «Видалити/Редагувати» в таблиці, якщо рядки генеруються динамічно? Що таке делегування подій і навіщо воно може бути потрібне?
15. Що таке `localStorage` (на рівні визначення) і для чого його можна використати в межах лабораторної роботи? Які обмеження цього механізму?